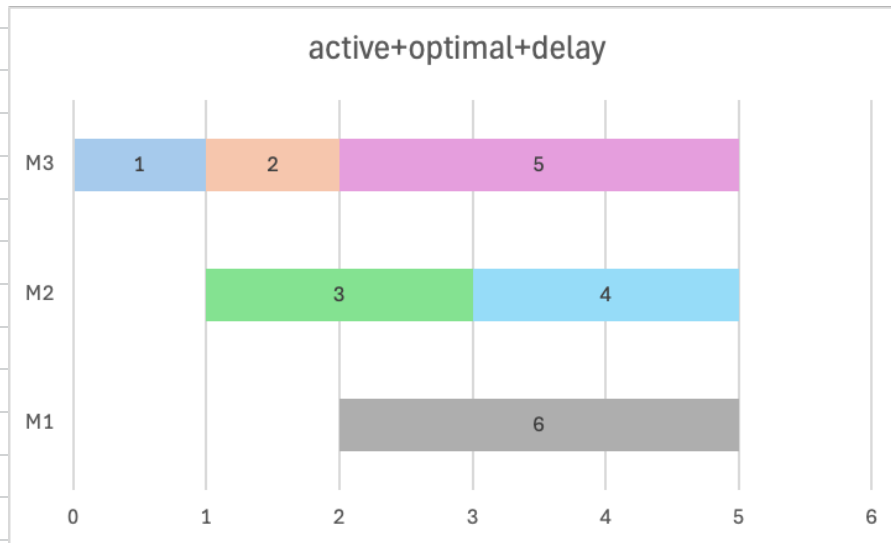


IDO6001 스케줄링이론 및 응용
2025-2 Homework 01
장서현 (2025324096)

1 Problem 1 (2 points)

활성 스케줄이면서 최적이지만 비지연 스케줄이 아닌 예제를 하나 만들어 간트 차트로 보이시오. 간트 차트를 바탕으로 예제의 활성 스케줄 여부, 최적 스케줄 여부, 비지연 스케줄 여부를 자세하게 설명하시오.



시점 0에 모두 대기 중인 작업이 6개가 있으며 각 작업이 선택해서 갈 수 있는 병렬기계가 3대 있고, 작업의 가공시간은 (1, 1, 2, 2, 3, 3)이다.

또한 작업 사이에는 다음과 같은 실행순서 (1->2, 1->3, 1->4, 2->5, 2->6)가 존재한다.

위의 스케줄은 활성이며, 최적이며, 비지연이 아닌 (지연) 스케줄이다.

1) 활성 스케줄 여부

: 활성 스케줄은 기계에서 작업의 순서를 변경하였을때 다른 작업의 완료 시점을 늦추지 않고는 앞으로 이동시킨 작업의 완료시점을 줄일 수 없는 스케줄을 의미한다. J4를 시점1에 M1에서 J6 보다 먼저 진행시킬 수 있으나 J6의 완료시점이 6으로 늘어난다. 따라서 위 스케줄을 활성스케줄이다.

2) 최적 스케줄 여부

: 가공 순서 조건에 의해 위 스케줄의 최소 makespan은 5이다. makespan을 최소화하는 스케줄이 최적 스케줄이라고 판단할때, 위 스케줄은 makespan이 5이므로 최적 스케줄이다.

3) 지연 스케줄 여부

: 비지연 스케줄을 각 기계에서 대기 중인 작업이 있을때 고의적으로 유휴시간을 발생키지 않는 스케줄을 의미한다. 위의 스케줄은 J1이 끝나자마자 M1에서 J4를 진행시킬 수 있지만, 고의적으로 지연을 시켰으므로 비지연 스케줄이 아닌 지연스케줄이다.

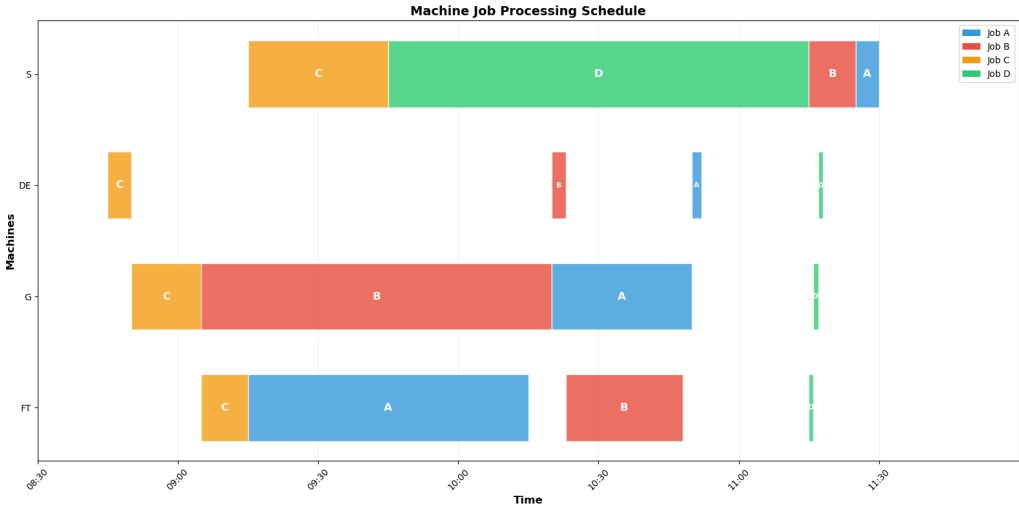
2 Problem 2 (3 points)

강의노트 1장 스케줄링개요 Example 1.1 연세 공작소 스케줄링 문제에 대해 활성 스케줄 생성 알고리즘을 적용하여 활성 스케줄을 작성하시오. Example 1.6의 표를 참고하여 알고리즘 단계별 진행과정을 상세하게 기술하고 활성 스케줄을 완성하시오. 단 Example 1.6의 예제와는 다른 활성 스케줄을 작성하고, 도출된 스케줄을 간트차트로 작성하시오 (간트차트는 되도록 프로그램을 통해 작성).

Table 1: 연세공작소 제품의 도착시간 및 가공순서와 가공시간

제품	도착시간	가공순서
A	8:30	FT(60) → G(30) → DE(2) → S(5)
B	8:45	G(75) → DE(3) → FT(25) → S(10)
C	8:45	DE(5) → G(15) → FT(10) → S(30)
D	9:30	S(90) → FT(1) → G(1) → DE(1)

단계 (t)	스케줄이 가능한 시간				$o_k \in S_t$	σ_k	Φ_k	Φ^*	스케줄 오퍼레이션
	FT	G	DE	S					σ_j
0	0	0	0	0	(A, FT)	08:30	09:30	-	-
					(B, G)	08:45	10:00	-	-
					(C, DE)	08:45	08:50	08:50	(C, DE)
					(D, S)	09:30	11:00	-	-
1	0	0	08:50	0	(A, FT)	08:30	09:30	-	-
					(B, G)	08:45	10:00	-	-
					(C, G)	08:50	09:05	09:05	(C, G)
					(D, S)	09:30	11:00	-	-
2	0	09:05	08:50	0	(A, FT)	08:30	09:30	-	-
					(B, G)	09:05	10:20	-	-
					(C, FT)	09:05	09:15	09:15	(C, FT)
					(D, S)	09:30	11:00	-	-
3	09:15	09:05	08:50	0	(A, FT)	09:15	10:15	-	-
					(B, G)	09:05	10:20	-	-
					(C, S)	09:15	09:45	09:45	(C, S)
					(D, S)	09:30	11:00	-	-
4	09:15	09:05	08:50	09:45	(A, FT)	09:15	10:15	10:15	(A, FT)
					(B, G)	09:05	10:20	-	-
					-	-	-	-	-
					(D, S)	09:45	11:15	-	-
5	10:15	09:05	08:50	09:45	(A, G)	10:15	10:45	-	-
					(B, G)	09:05	10:20	10:20	(B, G)
					-	-	-	-	-
					(D, S)	09:45	11:15	-	-
6	10:15	10:20	08:50	09:45	(A, G)	10:20	10:50	-	-
					(B, DE)	10:20	10:23	10:23	(B, DE)
					-	-	-	-	-
					(D, S)	09:45	11:15	-	-
7	10:15	10:20	10:23	09:45	(A, G)	10:20	10:50	-	-
					(B, FT)	10:23	10:48	10:48	(B, FT)
					-	-	-	-	-
					(D, S)	09:45	11:15	-	-
8	10:48	10:20	10:23	09:45	(A, G)	10:20	10:50	10:50	(A, G)
					(B, S)	10:48	10:58	-	-
					-	-	-	-	-
					(D, S)	09:45	11:15	-	-
9	10:48	10:50	10:23	09:45	(A, DE)	10:50	10:52	10:52	(A, DE)
					(B, S)	10:48	10:58	-	-
					-	-	-	-	-
					(D, S)	09:45	11:15	-	-
10	10:48	10:50	10:52	09:45	(A, S)	10:52	10:57	-	-
					(B, S)	10:48	10:58	-	-
					-	-	-	-	-
					(D, S)	09:45	11:15	11:15	(D, S)
11	10:48	10:50	10:52	11:15	(A, S)	11:15	11:20	-	-
					(B, S)	11:15	11:25	-	-
					-	-	-	-	-
					(D, FT)	11:15	11:16	11:16	(D, FT)
12	11:16	10:50	10:52	11:15	(A, S)	11:15	11:20	-	-
					(B, S)	11:15	11:25	-	-
					-	-	-	-	-
					(D, G)	11:16	11:17	11:17	(D, G)
13	11:16	11:17	10:52	11:15	(A, S)	11:15	11:20	-	-
					(B, S)	11:15	11:25	-	-
					-	-	-	-	-
					(D, DE)	11:17	11:18	11:18	(D, DE)
14	11:16	11:17	11:18	11:15	(A, S)	11:15	11:20	-	-
					(B, S)	11:15	11:25	11:25	(B, S)
					-	-	-	-	-
					-	-	-	-	-
15	11:16	11:17	11:18	11:25	(A, S)	11:25	11:30	11:30	(A, S)
					-	-	-	-	-
					-	-	-	-	-
					-	-	-	-	-



3 Problem 3 (3 points)

단일기계 환경의 생산라인에서 처리해야 할 작업이 4개인 상황을 고려해보자. 모든 작업의 가공시간은 5분으로 동일하다고 가정하자. 또한 이 문제에는 순서의존적(sequence-dependent) 작업준비시간(setup time)이 존재한다고 가정하자. 작업준비시간이란 작업이 완료된 후 다음 작업을 위해 기계를 정리하고 셋팅하는 시간이다. 이 문제에서는 작업준비시간이 고정된 시간이 아니라 스케줄상에서 선행 작업이 누구냐에 따라 바로 다음 작업의 준비시간이 아래 표와 같이 달라지는 것을 가정한다. 이 문제의 목적은 가공시간과 작업준비시간을 고려하여 작업이 모두 끝나는 시점,

		Table 2: 작업 준비 시간 후행 작업			
		1	2	3	4
선	1	-	3	5	9
행	2	4	-	2	8
작	3	3	3	-	6
업	4	2	7	1	-

즉 makespan을 최소화하는 것이다. 예를 들어 스케줄이 (1,2,3,4)이면 총 가공시간은 20이며 총 작업준비시간은 3+2+6=11이 되므로 작업이 완료되는 시점은 31이다. 이 문제를 다음과 같이 이웃 해 탐색 방법을 이용해서 풀고 그에 해당하는 스케줄을 제시하시오.

- 초기 해는 임의로(Random) 생성하시오.
- 이웃 해 생성 방법은 Single adjacent pairwise interchange(SAPI)와 Single pairwise interchange(SPI)를 사용하시오.
- 이웃 해 선택은 성능이 가장 좋은 이웃 해를 선택하는 방법(Greedy)과 임의로 선택하는 방법(Random)을 사용하시오.

이 세 가지 요소를 조합하면 4개의 이웃 해 탐색 방법이 만들어진다. 각 경우에 대해서 최대 5번씩 이웃 해 탐색을 반복하여 스케줄을 도출하고 비교하시오(컴퓨터 프로그램을 만들어 많이 반복하여 스케줄을 구해도 됨).

알고리즘 1: SAPI + Greedy 시뮬레이션

초기 해: [1, 3, 4, 2], Makespan: 38

알고리즘 2: SAPI + Random 시뮬레이션

초기 해: [1, 3, 4, 2], Makespan: 38

===== [반복 1] =====

현재 해: [1, 3, 4, 2] (Makespan: 38)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 4, 2], Makespan: 39
- 순서: [1, 4, 3, 2], Makespan: 33
- 순서: [1, 3, 2, 4], Makespan: 36

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [1, 4, 3, 2] (Makespan: 33)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [1, 4, 3, 2] (으)로 이동합니다.

===== [반복 1] =====

현재 해: [1, 3, 4, 2] (Makespan: 38)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 4, 2], Makespan: 39
- 순서: [1, 4, 3, 2], Makespan: 33
- 순서: [1, 3, 2, 4], Makespan: 36

[2] 이웃 해 선택 (random):

-> 무작위로 [1, 3, 2, 4] (Makespan: 36)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [1, 3, 2, 4] (으)로 이동합니다.

===== [반복 2] =====

현재 해: [1, 4, 3, 2] (Makespan: 33)

[1] 생성된 이웃 해 목록:

- 순서: [4, 1, 3, 2], Makespan: 30
- 순서: [1, 3, 4, 2], Makespan: 38
- 순서: [1, 4, 2, 3], Makespan: 38

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 1, 3, 2] (Makespan: 30)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [4, 1, 3, 2] (으)로 이동합니다.

===== [반복 2] =====

현재 해: [1, 3, 2, 4] (Makespan: 36)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 2, 4], Makespan: 34
- 순서: [1, 2, 3, 4], Makespan: 31
- 순서: [1, 3, 4, 2], Makespan: 38

[2] 이웃 해 선택 (random):

-> 무작위로 [1, 3, 4, 2] (Makespan: 38)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 3] =====

현재 해: [4, 1, 3, 2] (Makespan: 30)

[1] 생성된 이웃 해 목록:

- 순서: [1, 4, 3, 2], Makespan: 33
- 순서: [4, 3, 1, 2], Makespan: 27
- 순서: [4, 1, 2, 3], Makespan: 27

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 1, 2] (Makespan: 27)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [4, 3, 1, 2] (으)로 이동합니다.

===== [반복 3] =====

현재 해: [1, 3, 2, 4] (Makespan: 36)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 2, 4], Makespan: 34
- 순서: [1, 2, 3, 4], Makespan: 31
- 순서: [1, 3, 4, 2], Makespan: 38

[2] 이웃 해 선택 (random):

-> 무작위로 [1, 2, 3, 4] (Makespan: 31)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [1, 2, 3, 4] (으)로 이동합니다.

===== [반복 4] =====

현재 해: [4, 3, 1, 2] (Makespan: 27)

[1] 생성된 이웃 해 목록:

- 순서: [3, 4, 1, 2], Makespan: 31
- 순서: [4, 1, 3, 2], Makespan: 30
- 순서: [4, 3, 2, 1], Makespan: 28

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 2, 1] (Makespan: 28)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 4] =====

현재 해: [1, 2, 3, 4] (Makespan: 31)

[1] 생성된 이웃 해 목록:

- 순서: [2, 1, 3, 4], Makespan: 35
- 순서: [1, 3, 2, 4], Makespan: 36
- 순서: [1, 2, 4, 3], Makespan: 32

[2] 이웃 해 선택 (random):

-> 무작위로 [1, 2, 4, 3] (Makespan: 32)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 5] =====

현재 해: [4, 3, 1, 2] (Makespan: 27)

[1] 생성된 이웃 해 목록:

- 순서: [3, 4, 1, 2], Makespan: 31
- 순서: [4, 1, 3, 2], Makespan: 30
- 순서: [4, 3, 2, 1], Makespan: 28

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 2, 1] (Makespan: 28)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 5] =====

현재 해: [1, 2, 3, 4] (Makespan: 31)

[1] 생성된 이웃 해 목록:

- 순서: [2, 1, 3, 4], Makespan: 35
- 순서: [1, 3, 2, 4], Makespan: 36
- 순서: [1, 2, 4, 3], Makespan: 32

[2] 이웃 해 선택 (random):

-> 무작위로 [1, 3, 2, 4] (Makespan: 36)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

--- 최종 결과 ---

최종 해: [4, 3, 1, 2], Makespan: 27

--- 최종 결과 ---

최종 해: [1, 2, 3, 4], Makespan: 31

알고리즘 3: SPI + Greedy 시뮬레이션

초기 해: [1, 3, 4, 2], Makespan: 38

===== [반복 1] =====

현재 해: [1, 3, 4, 2] (Makespan: 38)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 4, 2], Makespan: 39
- 순서: [4, 3, 1, 2], Makespan: 27
- 순서: [2, 3, 4, 1], Makespan: 30
- 순서: [1, 4, 3, 2], Makespan: 33
- 순서: [1, 2, 4, 3], Makespan: 32
- 순서: [1, 3, 2, 4], Makespan: 36

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 1, 2] (Makespan: 27)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [4, 3, 1, 2](으)로 이동합니다.

===== [반복 2] =====

현재 해: [4, 3, 1, 2] (Makespan: 27)

[1] 생성된 이웃 해 목록:

- 순서: [3, 4, 1, 2], Makespan: 31
- 순서: [1, 3, 4, 2], Makespan: 38
- 순서: [2, 3, 1, 4], Makespan: 34
- 순서: [4, 1, 3, 2], Makespan: 30
- 순서: [4, 2, 1, 3], Makespan: 36
- 순서: [4, 3, 2, 1], Makespan: 28

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 2, 1] (Makespan: 28)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 3] =====

현재 해: [4, 3, 1, 2] (Makespan: 27)

[1] 생성된 이웃 해 목록:

- 순서: [3, 4, 1, 2], Makespan: 31
- 순서: [1, 3, 4, 2], Makespan: 38
- 순서: [2, 3, 1, 4], Makespan: 34
- 순서: [4, 1, 3, 2], Makespan: 30
- 순서: [4, 2, 1, 3], Makespan: 36
- 순서: [4, 3, 2, 1], Makespan: 28

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 2, 1] (Makespan: 28)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 4] =====

현재 해: [4, 3, 1, 2] (Makespan: 27)

[1] 생성된 이웃 해 목록:

- 순서: [3, 4, 1, 2], Makespan: 31
- 순서: [1, 3, 4, 2], Makespan: 38
- 순서: [2, 3, 1, 4], Makespan: 34
- 순서: [4, 1, 3, 2], Makespan: 30
- 순서: [4, 2, 1, 3], Makespan: 36
- 순서: [4, 3, 2, 1], Makespan: 28

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 2, 1] (Makespan: 28)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 5] =====

현재 해: [4, 3, 1, 2] (Makespan: 27)

[1] 생성된 이웃 해 목록:

- 순서: [3, 4, 1, 2], Makespan: 31
- 순서: [1, 3, 4, 2], Makespan: 38
- 순서: [2, 3, 1, 4], Makespan: 34
- 순서: [4, 1, 3, 2], Makespan: 30
- 순서: [4, 2, 1, 3], Makespan: 36
- 순서: [4, 3, 2, 1], Makespan: 28

[2] 이웃 해 선택 (greedy):

-> 가장 좋은 해인 [4, 3, 2, 1] (Makespan: 28)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

--- 최종 결과 ---

최종 해: [4, 3, 1, 2], Makespan: 27

알고리즘 4: SPI + Random 시뮬레이션

초기 해: [1, 3, 4, 2], Makespan: 38

===== [반복 1] =====

현재 해: [1, 3, 4, 2] (Makespan: 38)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 4, 2], Makespan: 39
- 순서: [4, 3, 1, 2], Makespan: 27
- 순서: [2, 3, 4, 1], Makespan: 30
- 순서: [1, 4, 3, 2], Makespan: 33
- 순서: [1, 2, 4, 3], Makespan: 32
- 순서: [1, 3, 2, 4], Makespan: 36

[2] 이웃 해 선택 (random):

-> 무작위로 [3, 1, 4, 2] (Makespan: 39)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 2] =====

현재 해: [1, 3, 4, 2] (Makespan: 38)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 4, 2], Makespan: 39
- 순서: [4, 3, 1, 2], Makespan: 27
- 순서: [2, 3, 4, 1], Makespan: 30
- 순서: [1, 4, 3, 2], Makespan: 33
- 순서: [1, 2, 4, 3], Makespan: 32
- 순서: [1, 3, 2, 4], Makespan: 36

[2] 이웃 해 선택 (random):

-> 무작위로 [1, 3, 2, 4] (Makespan: 36)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [1, 3, 2, 4](으)로 이동합니다.

===== [반복 3] =====

현재 해: [1, 3, 2, 4] (Makespan: 36)

[1] 생성된 이웃 해 목록:

- 순서: [3, 1, 2, 4], Makespan: 34
- 순서: [2, 3, 1, 4], Makespan: 34
- 순서: [4, 3, 2, 1], Makespan: 28
- 순서: [1, 2, 3, 4], Makespan: 31
- 순서: [1, 4, 2, 3], Makespan: 38
- 순서: [1, 3, 4, 2], Makespan: 38

[2] 이웃 해 선택 (random):

-> 무작위로 [3, 1, 2, 4] (Makespan: 34)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [3, 1, 2, 4](으)로 이동합니다.

===== [반복 4] =====

현재 해: [3, 1, 2, 4] (Makespan: 34)

[1] 생성된 이웃 해 목록:

- 순서: [1, 3, 2, 4], Makespan: 36
- 순서: [2, 1, 3, 4], Makespan: 35
- 순서: [4, 1, 2, 3], Makespan: 27
- 순서: [3, 2, 1, 4], Makespan: 36
- 순서: [3, 4, 2, 1], Makespan: 37
- 순서: [3, 1, 4, 2], Makespan: 39

[2] 이웃 해 선택 (random):

-> 무작위로 [3, 4, 2, 1] (Makespan: 37)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋지 않으므로 이동하지 않습니다. 현재 해를 유지합니다.

===== [반복 5] =====

현재 해: [3, 1, 2, 4] (Makespan: 34)

[1] 생성된 이웃 해 목록:

- 순서: [1, 3, 2, 4], Makespan: 36
- 순서: [2, 1, 3, 4], Makespan: 35
- 순서: [4, 1, 2, 3], Makespan: 27
- 순서: [3, 2, 1, 4], Makespan: 36
- 순서: [3, 4, 2, 1], Makespan: 37
- 순서: [3, 1, 4, 2], Makespan: 39

[2] 이웃 해 선택 (random):

-> 무작위로 [4, 1, 2, 3] (Makespan: 27)을(를) 선택.

[3] 이동 결정:

-> 현재 해보다 좋으므로 [4, 1, 2, 3](으)로 이동합니다.

--- 최종 결과 ---

최종 해: [4, 1, 2, 3], Makespan: 27

해 선택 방식: Greedy vs Random

- Greedy Search는 언제나 가장 좋은 이웃해를 선택하기 때문에 안정적으로 성능을 개선하는 경향을 보이고, 이번 시뮬레이션 결과 두 Greedy 알고리즘 모두 최적 Makespan 27을 찾아냈다.
- Random Search는 이웃해 선택을 랜덤하게 하므로 성능 개선이 불안정하다. 그 결과 SAPI+Random은 초기해보다 나쁜 결과를 냈고, SPI+Random은 최적해를 찾았다.

이웃해 생성 방식: SAPI vs SPI

- SAPI는 탐색 범위가 좁아 점진적인 개선을 보인다
- SPI는 탐색 범위가 넓어 더 빠른 개선을 보인다. SPI+Greedy는 첫 iteration에 최적해를 찾았다.

결론

- 더 넓은 범위의 탐색과 안정적인 성능 개선이 가능한 SPI + Greedy 알고리즘이 가장 robust할 것으로 예상된다.

4 Problem 4 (2 points)

성능지표 C 와 C_{max} 는 equivalent 하지 않다는 것을 예제를 들어 증명하시오.

시점 0에 모두 대기 중인 작업이 3개가 있으며 각 작업이 선택해서 갈 수 있는 병렬기계가 2대 있고, 작업의 가공시간은 (8, 7, 2)이다.

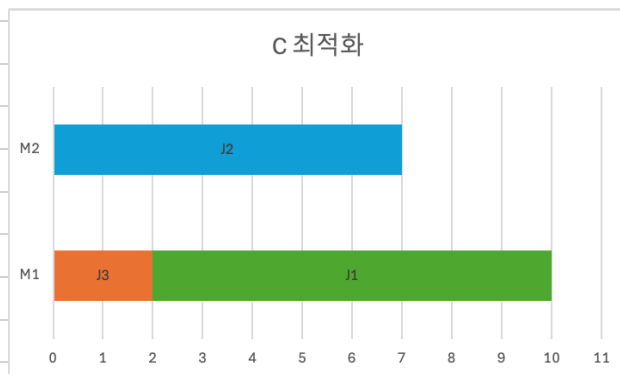
1. C 평균 완료 시간 최적화

각 작업이 시스템에 머무는 평균 시간을 최소화하기 위해서 SPT 규칙을 적용한다.

M1: J3 (2) → J1 (8)

M2: J2 (7)

J1의 완료시간은 10, J2의 완료시간 7, J3의 완료시간 2로 C 는 6.33이며, C_{max} 는 10이다.



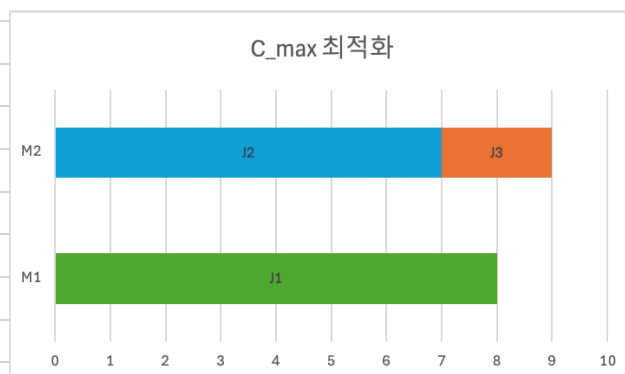
2. C_{max} 모든 작업의 완료시점 최적화

C_{max} 최적화를 위해 두 기계의 작업시간을 균형있게 분배한 스케줄이다

M1: J1 (8)

M2: J3 (7) → J2 (2)

J1의 완료시간은 8, J2의 완료시간은 9, J3의 완료시간은 7로 C 는 8이며, C_{max} 는 9이다.



위 예시에서 볼 수 있듯, C_{max} 를 최적화하는 스케줄을 C 를 최적화하지 못하고, 반대로 C 를 최적화하는 스케줄은 C_{max} 를 최적화하지 못한다.

따라서, 두 성능지표 C 와 C_{max} 는 equivalent하지 않다.

5 Problem 5 (5 points)

성능지표 $C_{\max}, F, F_{\max}, L, L_{\max}, T, T_{\max}$ 는 regular measure임을 증명하시오.

기본가정 : 두개의 스케줄 S 와 S' 이 있을때, 모든 작업 j 에 대해 $C_j \leq C'_j$ 이 성립한다.

① $C_{\max}$

스케줄 S 의 $C_{\max}$ 는 $\{C_1, C_2, \dots, C_n\}$ 집합의 최댓값.

스케줄 S' 의 $C_{\max}$ 는 $\{C'_1, C'_2, \dots, C'_n\}$ 집합의 최댓값.

S 의 집합은 S' 집합보다 모든 원소가 크거나 같다.

따라서 $\max\{C_1, \dots, C_n\} \leq \max\{C'_1, \dots, C'_n\}$ 은 항상 성립한다.

$\therefore C_{\max} \leq C'_{\max} \rightarrow C_{\max}$ 는 regular measure.

② F

$C_j \leq C'_j$ 이라면 $\sum_{j=1}^n C_j \leq \sum_{j=1}^n C'_j$ 도 성립한다.

$$\sum_{j=1}^n C_j - \sum_{j=1}^n r_j \leq \sum_{j=1}^n C'_j - \sum_{j=1}^n r_j \quad \text{또한 성립한다. } (r_j \text{는 정해진 상수})$$

정의에 의해, $\sum_{j=1}^n F_j \leq \sum_{j=1}^n F'_j$ 으로 바꿀 수 있다.

$\therefore F \leq F' \rightarrow F$ 는 regular measure

③ $F_{\max}$

F 는 regular measure 이기에 $F \leq F'$ 이 성립한다.

따라서 각 스케줄에서 가장 큰 흐름시간을 선택해도 이 관계는 유지된다.

$$\max\{F_1, \dots, F_n\} \leq \max\{F'_1, \dots, F'_n\}$$

$\therefore F_{\max} \leq F'_{\max} \rightarrow F_{\max}$ 는 regular measure.

④ L

$C_j \leq C'_j$ 이라면 $\sum_{j=1}^n C_j \leq \sum_{j=1}^n C'_j$ 도 성립한다.

$$\sum_{j=1}^n C_j - \sum_{j=1}^n d_j \leq \sum_{j=1}^n C'_j - \sum_{j=1}^n d_j \quad \text{또한 성립한다. } (d_j \text{는 정해진 상수})$$

정의에 의해, $\sum_{j=1}^n L_j \leq \sum_{j=1}^n L'_j$ 으로 바꿀 수 있다.

$\therefore L \leq L' \rightarrow L$ 는 regular measure

⑤ $L_{\max}$

L 는 regular measure 이기에 $L \leq L'$ 이 성립한다.

따라서 각 스케줄에서 가장 큰 지연시간을 선택해도 이 관계는 유지된다.

$$\max\{L_1, \dots, L_n\} \leq \max\{L'_1, \dots, L'_n\}$$

$\therefore L_{\max} \leq L'_{\max} \rightarrow L_{\max}$ 는 regular measure.

⑥ T

$T_j = \max\{L_j, 0\}$, $T'_j = \max\{L'_j, 0\}$ 이라고 정의한 수 있다.

• $L'_j \leq 0$ 이라면, L 는 regular measure 임으로 $L_j \leq L'_j \leq 0$ 이다.
 $\therefore T_j = T'_j = 0$ 이다

• $L'_j > 0$ 이라면, $T'_j = L'_j$ 이다.

$T_j \leq L_j$ 일것이고, $L_j \leq L'_j$ 일것이다.

$\therefore T_j \leq L_j \leq L'_j = T'_j$ 이다.

따라서 $T_j \leq T'_j$, $\sum T_j \leq \sum T'_j$, $T \leq T'$ 이다.

$\therefore T \leq T' \rightarrow T$ 는 regular measure

⑦ $T_{\max}$

T 는 regular measure 이기에 $T \leq T'$ 이 성립한다.

따라서 각 스케줄에서 가장 큰 위반시간을 선택해도 이 관계는 유지된다.

$$\max\{T_1, \dots, T_n\} \leq \max\{T'_1, \dots, T'_n\}$$

$\therefore T_{\max} \leq T'_{\max} \rightarrow T_{\max}$ 는 regular measure.